

Elementary Programming

Java Syntax

- In the previous chapter, we created a Java file called Main.java, and we used the following code to print "Hello World" to the screen:

```
Main.java // file name
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

Explanation of example

- Every line of code that runs in Java must be inside a class. The class name should always start with an uppercase first letter.
 - In our example, we named the class **Main**.
- **Note:** Java is case-sensitive. **MyClass** and **myclass** would be treated as two completely different names.
- The name of the Java file **must match** the class name. So if your class is called Main, the file must be saved as Main.java. This is because Java uses the class name to find and run your code. If the names don't match, Java will give an error and the program will not run.
- When saving the file, save it using the class name and add .java to the end of the filename

The main Method

The `main()` method is required in every Java program. It is where the program starts running:

```
public static void main(String[] args)
```

Any code placed inside the `main()` method will be executed.

For now, you need to understand the keywords `public`, `static`, and `void`. remember: `main()` is the starting point of every Java program.

System.out.println()

Inside the main() method, we can use the println() method to print a line of text to the screen:

Example

```
public static void main(String[] args) {  
    System.out.println("Hello World");  
}
```

Note: The curly braces `{ }` mark the beginning and the end of a block of code.

`System.out.println()` may look long, but you can think of it as a single command that means:

"Send this text to the screen."

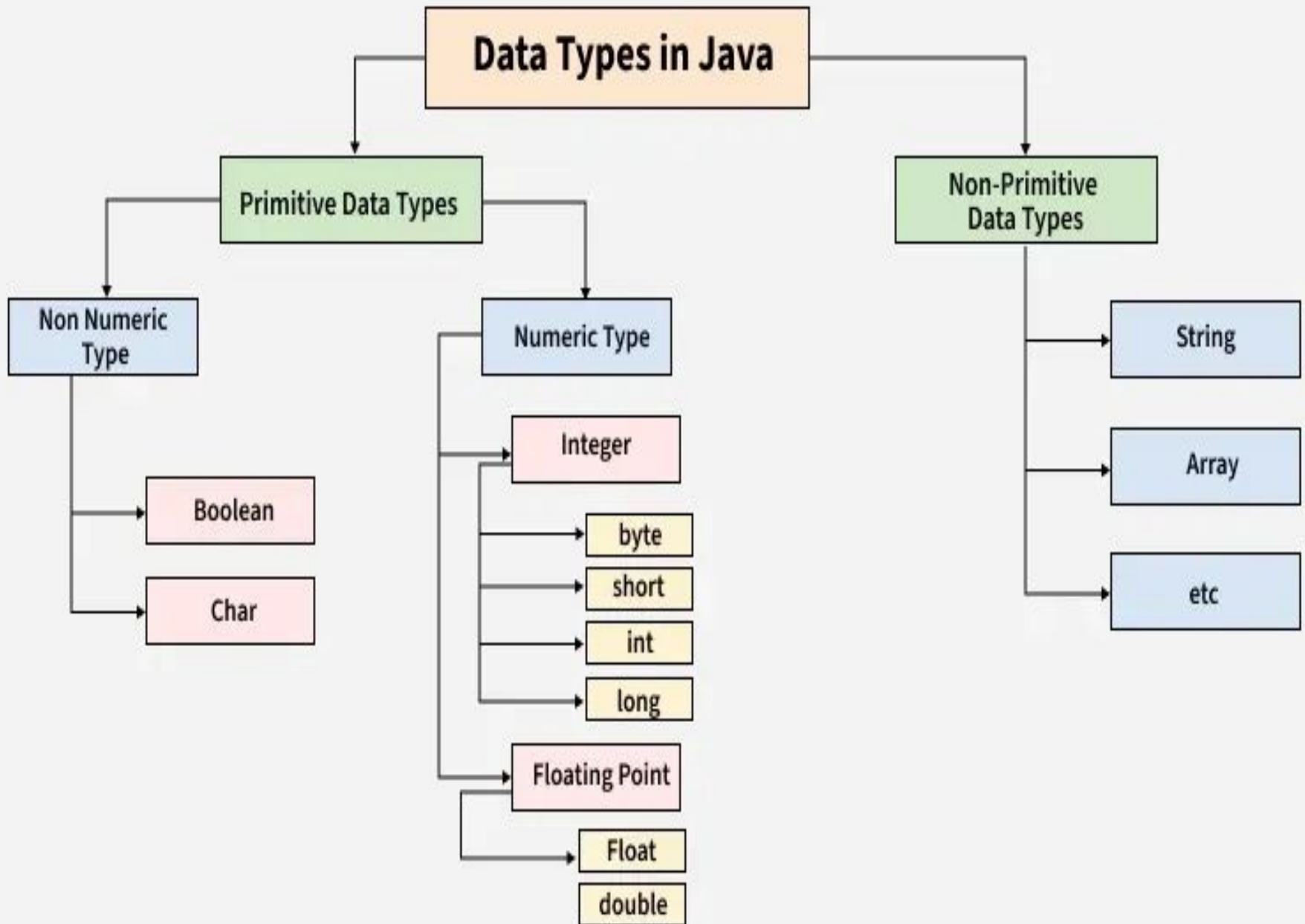
Here's what each part means (you will learn the details later):

- `System` is a built-in Java class.
- `out` is a member of `System`, short for "output".
- `println()` is a method, short for "print line".

Finally, remember that each Java statement must end with a semicolon `;`.

Data type

- The kind of data that a variable can hold.
- Java is strongly typed – a variable can be only one data type and only values of that type are permitted.



The 8 primitive data Java types

- **byte** – smallest integer type
- **short** – 2 byte integer type
- **int** – 4 byte integer type
- **long** – 8 byte integer type
- **float** – a 7 significant digit floating point type
- **double** – a 15 significant digit floating point type
- **char** – represents a single Unicode character
- **boolean** – represents true or false values

Declaring a variable

A **declaration** is a Java statement that builds a container in the program. It associates a name and data type with an area of memory.

Datatype variable;

Examples:

```
int count;  
double total;  
char firstLetter;
```

Initializing a variable

- **Initialization** is the process of giving a variable its first value.
- The value must be consistent with the type of the variable.
- A value may be a literal, another variable, or an expression (calculation).
- A **literal** is a value that is written directly into the program.

Let's look at a demo

- Variables.java

Trace a Program Execution

```
public class ComputeArea {  
    /** Main method */  
    public static void main(String[] args) {
```

```
        double radius;
```

```
        double area;
```

```
        // Assign a radius  
        radius = 20;
```

```
        // Compute area  
        area = radius * radius * 3.14159;
```

```
        // Display results  
        System.out.println("The area for the circle of radius " +  
            radius + " is " + area);
```

```
    }  
}
```

allocate memory
for radius

radius

no value

Trace a Program Execution

```
public class ComputeArea {  
    /** Main method */  
    public static void main(String[] args) {  
        double radius;  
        double area;
```

```
        // Assign a radius  
        radius = 20;
```

```
        // Compute area  
        area = radius * radius * 3.14159;
```

```
        // Display results  
        System.out.println("The area for the circle of radius " +  
            radius + " is " + area);  
    }  
}
```

memory

radius

no value

area

no value

allocate memory
for area

Trace a Program Execution

```
public class ComputeArea {  
    /** Main method */  
    public static void main(String[] args) {  
        double radius;  
        double area;
```

```
        // Assign a radius
```

```
        radius = 20;
```

```
        // Compute area
```

```
        area = radius * radius * 3.14159;
```

```
        // Display results
```

```
        System.out.println("The area for the circle of radius " +  
            radius + " is " + area);
```

```
    }
```

```
}
```

assign 20 to radius

radius

20

area

no value

Trace a Program Execution

```
public class ComputeArea {  
    /** Main method */  
    public static void main(String[] args) {  
        double radius;  
        double area;  
  
        // Assign a radius  
        radius = 20;  
  
        // Compute area  
        area = radius * radius * 3.14159;  
  
        // Display results  
        System.out.println("The area for the circle of radius " +  
            radius + " is " + area);  
    }  
}
```

memory	
radius	20
area	1256.636

compute area and assign it
to variable area

Trace a Program Execution

```
public class ComputeArea {  
    /** Main method */  
    public static void main(String[] args) {  
        double radius;  
        double area;  
  
        // Assign a radius  
        radius = 20;  
  
        // Compute area  
        area = radius * radius * 3.14159;  
  
        // Display results  
        System.out.println("The area for the circle of radius " +  
            radius + " is " + area);  
    }  
}
```

memory

radius

20

area

1256.636

print a message to the
console



```
CA\ Command Prompt  
c:\book>java ComputeArea  
The area for the circle of radius 20.0 is 1256.636
```

Reading Input from the Console

1. Create a Scanner object

```
Scanner input = new Scanner(System.in);
```

2. Use the method nextDouble() to obtain to a double value.
For example,

```
System.out.print("Enter a double value: ");  
Scanner input = new Scanner(System.in);  
double d = input.nextDouble();
```

ComputeAreaWithConsoleInput

Run

ComputeAverage

Run

Identifiers

- An identifier is a sequence of characters that consist of letters, digits, underscores (`_`), and dollar signs (`$`).
- An identifier must start with a letter, an underscore (`_`), or a dollar sign (`$`). It cannot start with a digit.
- An identifier cannot be a reserved word. (See Appendix A, “Java Keywords,” for a list of reserved words).
- An identifier cannot be `true`, `false`, or `null`.
- An identifier can be of any length.

Variables

```
// Compute the first area  
radius = 1.0;  
area = radius * radius * 3.14159;  
System.out.println("The area is " +  
    area + " for radius "+radius);
```

```
// Compute the second area  
radius = 2.0;  
area = radius * radius * 3.14159;  
System.out.println("The area is " +  
    area + " for radius "+radius);
```

Declaring Variables

```
int x;           // Declare x to be an
                  // integer variable;

double radius;  // Declare radius to
                  // be a double variable;

char a;          // Declare a to be a
                  // character variable;
```

Assignment Statements

```
x = 1;           // Assign 1 to x;  
radius = 1.0;    // Assign 1.0 to radius;  
a = 'A';         // Assign 'A' to a;
```

Declaring and Initializing in One Step

- `int x = 1;`
- `double d = 1.4;`

Named Constants

```
final datatype CONSTANTNAME = VALUE;
```

```
final double PI = 3.14159;
```

```
final int SIZE = 3;
```


Naming Conventions

- Choose meaningful and descriptive names.
- Variables and method names:
 - Use lowercase. If the name consists of several words, concatenate all in one, use lowercase for the first word, and capitalize the first letter of each subsequent word in the name.
 - For example, the variables `radius` and `area`, and the method `computeArea`.

Naming Conventions, cont.

- Class names:
 - Capitalize the first letter of each word in the name. For example, the class name `ComputeArea`.
- Constants:
 - Capitalize all letters in constants, and use underscores to connect words. For example, the constant `PI` and `MAX_VALUE`

Numerical Data Types

Name	Range	Storage Size
byte	-2^7 to $2^7 - 1$ (-128 to 127)	8-bit signed
short	-2^{15} to $2^{15} - 1$ (-32768 to 32767)	16-bit signed
int	-2^{31} to $2^{31} - 1$ (-2147483648 to 2147483647)	32-bit signed
long	-2^{63} to $2^{63} - 1$ (i.e., -9223372036854775808 to 9223372036854775807)	64-bit signed
float	Negative range: -3.4028235E+38 to -1.4E-45 Positive range: 1.4E-45 to 3.4028235E+38	32-bit IEEE 754
double	Negative range: -1.7976931348623157E+308 to -4.9E-324 Positive range: 4.9E-324 to 1.7976931348623157E+308	64-bit IEEE 754

Reading Numbers from the Keyboard

```
Scanner input = new Scanner(System.in) ;  
int value = input.nextInt() ;
```

Method	Description
<code>nextByte()</code>	reads an integer of the <code>byte</code> type.
<code>nextShort()</code>	reads an integer of the <code>short</code> type.
<code>nextInt()</code>	reads an integer of the <code>int</code> type.
<code>nextLong()</code>	reads an integer of the <code>long</code> type.
<code>nextFloat()</code>	reads a number of the <code>float</code> type.
<code>nextDouble()</code>	reads a number of the <code>double</code> type.

Numeric Operators

Name	Meaning	Example	Result
+	Addition	34 + 1	35
-	Subtraction	34.0 - 0.1	33.9
*	Multiplication	300 * 30	9000
/	Division	1.0 / 2.0	0.5
%	Remainder	20 % 3	2

Integer Division

+, -, *, /, and %

5 / 2 yields an integer 2.

5.0 / 2 yields a double value 2.5

5 % 2 yields 1 (the remainder of the division)

Remainder Operator

- Remainder is very useful in programming.
 - For example, an even number % 2 is always 0 and an odd number % 2 is always 1.

So you can use this property to determine whether a number is even or odd.

Suppose today is Saturday and you and your friends are going to meet in 10 days. What day is in 10 days? You can find that day is Tuesday using the following expression:

Saturday is the 6th day in a week

$(6 + 10) \% 7$ is 2

A week has 7 days

The 2nd day in a week is Tuesday

After 10 days

Problem: Displaying Time

Write a program that obtains minutes and remaining seconds from seconds.

DisplayTime

Run

NOTE

Calculations involving floating-point numbers are approximated because these numbers are not stored with complete accuracy. For example,

```
System.out.println(1.0 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1);
```

displays 0.50000000000000000001, not 0.5, and

```
System.out.println(1.0 - 0.9);
```

displays 0.099999999999999999998, not 0.1. Integers are stored precisely. Therefore, calculations with integers yield a precise integer result.

Exponent Operations

```
System.out.println(Math.pow(2, 3));
```

```
// Displays 8.0
```

```
System.out.println(Math.pow(4, 0.5));
```

```
// Displays 2.0
```

```
System.out.println(Math.pow(2.5, 2));
```

```
// Displays 6.25
```

```
System.out.println(Math.pow(2.5, -2));
```

```
// Displays 0.16
```

Number Literals

A *literal* is a constant value that appears directly in the program. For example, 34, 1,000,000, and 5.0 are literals in the following statements:

```
int i = 34;
```

```
long x = 1000000;
```

```
double d = 5.0;
```

Integer Literals

- An integer literal can be assigned to an integer variable as long as it can fit into the variable.
- A compilation error would occur if the literal were too large for the variable to hold.
 - For example, the statement `byte b = 1000` would cause a compilation error, because 1000 cannot be stored in a variable of the byte type.
- An integer literal is assumed to be of the `int` type, whose value is between -2^{31} (-2147483648) to $2^{31}-1$ (2147483647).
- To denote an integer literal of the long type, append it with the letter `L` or `l`. `L` is preferred because `l` (lowercase `L`) can easily be confused with `1` (the digit one).

Floating-Point Literals

- Floating-point literals are written with a decimal point. By default, a floating-point literal is treated as a double type value.
 - For example, 5.0 is considered a double value, not a float value. You can make a number a float by appending the letter f or F, and make a number a double by appending the letter d or D.
 - For example, you can use 100.2f or 100.2F for a float number, and 100.2d or 100.2D for a double number.

double vs. float

The double type values are more accurate than the float type values. For example,

```
System.out.println("1.0 / 3.0 is " + 1.0 / 3.0);
```

displays `1.0 / 3.0 is 0.3333333333333333`

16 digits

```
System.out.println("1.0F / 3.0F is " + 1.0F / 3.0F);
```

displays `1.0F / 3.0F is 0.33333334`

7 digits

Scientific Notation

Floating-point literals can also be specified in scientific notation, for example, `1.23456e+2`, same as `1.23456e2`, is equivalent to `123.456`, and `1.23456e-2` is equivalent to `0.0123456`. E (or e) represents an exponent and it can be either in lowercase or uppercase.

Arithmetic Expressions

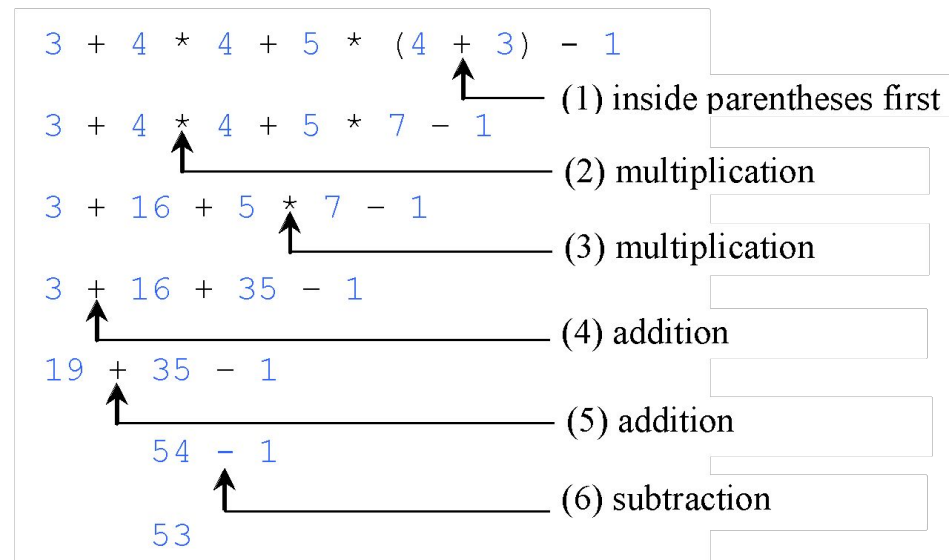
$$\boxed{\frac{3+4x}{5} - \frac{10(y-5)(a+b+c)}{x} + 9\left(\frac{4}{x} + \frac{9+x}{y}\right)}$$

is translated to

$$(3+4*x)/5 - 10*(y-5)*(a+b+c)/x + 9*(4/x + (9+x)/y)$$

How to Evaluate an Expression

Though Java has its own way to evaluate an expression behind the scene, the result of a Java expression and its corresponding arithmetic expression are the same. Therefore, you can safely apply the arithmetic rule for evaluating a Java expression.



Problem: Converting Temperatures

Write a program that converts a Fahrenheit degree to Celsius using the formula:

$$celsius = (\frac{5}{9})(fahrenheit - 32)$$

Note: you have to write

$$celsius = (5.0 / 9) * (fahrenheit - 32)$$

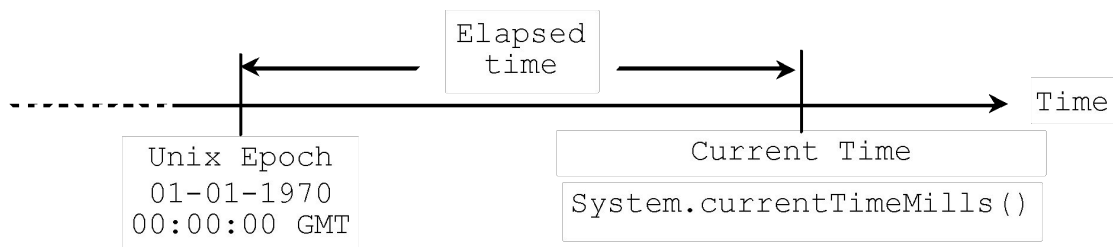
FahrenheitToCelsius

Run

Problem: Displaying Current Time

Write a program that displays current time in GMT in the format hour:minute:second such as 1:45:19.

The `currentTimeMillis` method in the `System` class returns the current time in milliseconds since the midnight, January 1, 1970 GMT. (1970 was the year when the Unix operating system was formally introduced.) You can use this method to obtain the current time, and then compute the current second, minute, and hour as follows.



ShowCurrentTime

Run

Augmented Assignment Operators

<i>Operator</i>	<i>Name</i>	<i>Example</i>	<i>Equivalent</i>
<code>+=</code>	Addition assignment	<code>i += 8</code>	<code>i = i + 8</code>
<code>-=</code>	Subtraction assignment	<code>i -= 8</code>	<code>i = i - 8</code>
<code>*=</code>	Multiplication assignment	<code>i *= 8</code>	<code>i = i * 8</code>
<code>/=</code>	Division assignment	<code>i /= 8</code>	<code>i = i / 8</code>
<code>%=</code>	Remainder assignment	<code>i %= 8</code>	<code>i = i % 8</code>

Increment and Decrement Operators

<i>Operator</i>	<i>Name</i>	<i>Description</i>	<i>Example (assume i = 1)</i>
++var	preincrement	Increment var by 1 , and use the new var value in the statement	int j = ++i; // j is 2, i is 2
var++	postincrement	Increment var by 1 , but use the original var value in the statement	int j = i++; // j is 1, i is 2
--var	predecrement	Decrement var by 1 , and use the new var value in the statement	int j = --i; // j is 0, i is 0
var--	postdecrement	Decrement var by 1 , and use the original var value in the statement	int j = i--; // j is 1, i is 0

Increment and Decrement Operators, cont.

```
int i = 10;  
int newNum = 10 * i++;
```

Same effect as

```
int newNum = 10 * i;  
i = i + 1;
```

```
int i = 10;  
int newNum = 10 * (++i);
```

Same effect as

```
i = i + 1;  
int newNum = 10 * i;
```

Increment and Decrement Operators, cont.

Using increment and decrement operators makes expressions short, but it also makes them complex and difficult to read. Avoid using these operators in expressions that modify multiple variables, or the same variable for multiple times such as this: `int k = ++i + i.`

Assignment Expressions and Assignment Statements

Prior to Java 2, all the expressions can be used as statements. Since Java 2, only the following types of expressions can be statements:

`variable op= expression; // Where op is +, -, *, /, or %`

`++variable;`

`variable++;`

`--variable;`

`variable--;`

Numeric Type Conversion

Consider the following statements:

```
byte i = 100;
```

```
long k = i * 3 + 4;
```

```
double d = i * 3.1 + k / 2;
```

Conversion Rules

When performing a binary operation involving two operands of different types, Java automatically converts the operand based on the following rules:

1. If one of the operands is double, the other is converted into double.
2. Otherwise, if one of the operands is float, the other is converted into float.
3. Otherwise, if one of the operands is long, the other is converted into long.
4. Otherwise, both operands are converted into int.

Type Casting

Implicit casting

```
double d = 3; (type widening)
```

Explicit casting

```
int i = (int)3.0; (type narrowing)
```

```
int i = (int)3.9; (Fraction part is truncated)
```

What is wrong? `int x = 5 / 2.0;`

range increases

byte, short, int, long, float, double

Casting in an Augmented Expression

In Java, an augmented expression of the form **x1 op= x2** is implemented as **x1 = (T)(x1 op x2)**, where **T** is the type for **x1**. Therefore, the following code is correct.

```
int sum = 0;
```

```
sum += 4.5; // sum becomes 4 after this statement
```

```
sum += 4.5 is equivalent to sum = (int)(sum + 4.5).
```

Common Errors and Pitfalls

- Common Error 1: Undeclared/Uninitialized Variables and Unused Variables
- Common Error 2: Integer Overflow
- Common Error 3: Round-off Errors
- Common Error 4: Unintended Integer Division
- Common Error 5: Redundant Input Objects

- Common Pitfall 1: Redundant Input Objects

Common Error 1: Undeclared/Uninitialized Variables and Unused Variables

```
double interestRate = 0.05;  
double interest = interestrate * 45;
```

Common Error 2: Integer Overflow

```
int value = 2147483647 + 1;
```

```
// value will actually be -2147483648
```

Common Error 3: Round-off Errors

```
System.out.println(1.0 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1);
```

```
System.out.println(1.0 - 0.9);
```


Common Error 4: Unintended Integer Division

```
int number1 = 1;  
int number2 = 2;  
double average = (number1 + number2) / 2;  
System.out.println(average);
```

(a)

```
int number1 = 1;  
int number2 = 2;  
double average = (number1 + number2) / 2.0;  
System.out.println(average);
```

(b)

Common Pitfall 1: Redundant Input Objects

```
Scanner input = new Scanner(System.in);  
System.out.print("Enter an integer: ");  
int v1 = input.nextInt();
```

```
Scanner input1 = new Scanner(System.in);  
System.out.print("Enter a double value: ");  
double v2 = input1.nextDouble();
```

Types of Variable

- There are three types of variables in java:
 - local variable
 - instance variable
 - static variable

Local Variable

- A variable which is declared inside the method is called local variable.

instance variable

- A variable which is declared inside the class but outside the method, is called instance variable . It is not declared as static.

static variable

- A variable that is declared as static is called static variable. It cannot be local.

- Example to understand the types of variables in java

```
class A {  
    int data=50;//instance variable  
    static int m=100;//static variable  
    void method(){  
        int n=90;//local variable  
    }  
} //end of class
```

Scope of Variables

- **Scope** refers to *where a variable can be accessed* in a program.
- The scope of a variable defines the section of the code in which the variable is visible. As a general rule, variables that are defined within a block are not accessible outside that block
 - 1. Local Scope
 - 2. Block Scope
 - 3. Global Scope
 - 4. Function Parameters

Local Scope

- A variable declared inside a function or block {}.

```
void myFunction() {  
    int x = 10; // Local variable  
}
```

- **Accessible only inside myFunction**
- **Cannot be used outside the function**

Block Scope

- A variable declared inside a block like if, for, etc.

```
if (true) {  
    int y = 5;  
}
```

- **y exists only inside the if block**

Global Scope

- A variable declared outside all functions.

```
int globalVar = 100;
```

```
void myFunction() {  
    printf("%d", globalVar);  
}
```

- **Accessible from anywhere in the file**

Function Parameters

- Parameters are local to the function.

```
void add(int a, int b) {  
    int sum = a + b;  
}
```

- **a, b, and sum exist only inside add**

Lifetime of Variables

- **Lifetime** refers to *how long a variable exists in memory* during program execution.
- The lifetime of a variable refers to how long the variable exists before it is destroyed.
- Destroying variables refers to de-allocating the memory that was allotted to the variables when declaring it.
 - **Automatic Variables**
 - Static Variables
 - Global Variables

Automatic Variables

- Created when function starts
- Destroyed when function ends

```
void test() {  
    int x = 5; // Created here  
}           // Destroyed here
```

Static Variables

- Retain their value between function calls
- Exist for entire program duration

```
void counter() {  
    static int count = 0;  
    count++;  
    printf("%d\n", count);  
}
```

- If called multiple times:

1
2
3

Operators in java

- Operator in java is a symbol that is used to perform operations. For example: +, -, *, / etc.
- There are many types of operators in java which are given below:
 - Unary Operator,
 - Arithmetic Operator,
 - shift Operator,
 - Relational Operator,
 - Bitwise Operator,
 - Logical Operator,
 - Ternary Operator and
 - Assignment Operator.

Operators Hierarchy

Operator Precedence

Operators	Precedence
postfix	<code>expr++ expr--</code>
unary	<code>++expr --expr +expr -expr ~ !</code>
multiplicative	<code>* / %</code>
additive	<code>+ -</code>
shift	<code><< >> >>></code>
relational	<code>< > <= >= instanceof</code>
equality	<code>== !=</code>
bitwise AND	<code>&</code>
bitwise exclusive OR	<code>^</code>
bitwise inclusive OR	<code> </code>
logical AND	<code>&&</code>
logical OR	<code> </code>
ternary	<code>? :</code>
assignment	<code>= += -= *= /= %= &= ^= = <<= >>= >>>=</code>

Expressions

- Expressions are essential building blocks of any Java program, usually created to produce a new value, although sometimes an expression simply assigns a value to a variable. Expressions are built using values, variables, operators and method calls.

Types of Expressions

- While an expression frequently produces a result, it doesn't always. There are three types of expressions in Java:
 - Those that produce a value, i.e. the result of $(1 + 1)$
 - Those that assign a variable, for example $(v = 10)$
 - Those that have no result but might have a "side effect" because an expression can include a wide range of elements such as method invocations or increment operators that modify the state (i.e. memory) of a program.

Java Type casting and Type conversion

- Widening or Automatic Type Conversion
 - Widening conversion takes place when two data types are automatically converted. This happens when:
 - The two data types are compatible.
 - When we assign value of a smaller data type to a bigger data type.
 - For Example, in java the numeric data types are compatible with each other but no automatic conversion is supported from numeric type to char or boolean. Also, char and boolean are not compatible with each other.

Narrowing or Explicit Conversion

- If we want to assign a value of larger data type to a smaller data type we perform explicit type casting or narrowing. [?]
- This is useful for incompatible data types where automatic conversion cannot be done. [?] Here, target-type specifies the desired type to convert the specified value to.